MTH129 Introduction to AI for Mathematical Sciences (MS)

2025-2026 Academic Year
Semester 2

SMP AI Team

Xi'an Jiaotong-Liverpool University
西交利物浦大学

Neural Network Implementation

Table of Contents

1 Introduction

2 Implement Neural Networks
- Weights and Biases
- Activation Functions
- Output Layer

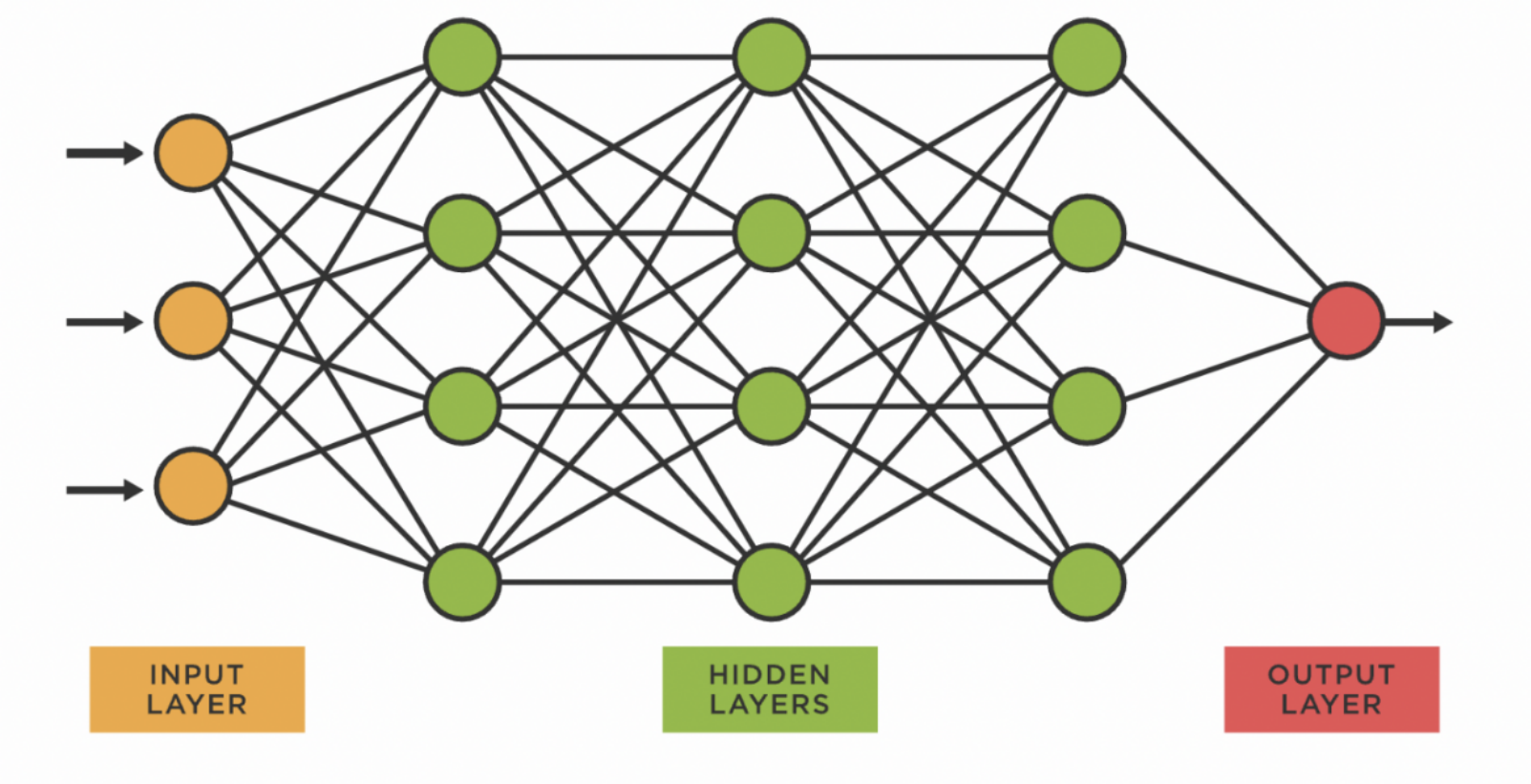
3 Neural Network Regression

4 Neural Network Classification

What is a Neural Network?

- Computational model inspired by biological neural networks
- Key components:
  - Input Layer: receives input features.
  - Weights and Biases: trainable parameters that adjust during learning.
  - Activation Functions: introduces non-linearity which allows the network to learn complex patterns.
  - Hidden Layers: process information through weighted connections and activation functions.
  - Output Layer: produces the final prediction.

Neural Network Illustration



Understanding the Input Layer

Input Layer Characteristics

- **Structure**
  - Number of nodes equals number of features
  - No computation performed (passive layer)
  - Direct connection to first hidden layer
- **Example: Regression (Housing Price Prediction)**
  - Input features: [MedInc, HouseAge, AveRooms, ...]
  - Input shape: (n_samples, 8 features)
  - Each feature normalized using StandardScaler
- **Example: Classification (Binary)**
  - Input features: [Feature1, Feature2]
  - Input shape: (n_samples, 2 features)
  - Features typically standardized to mean=0, std=1

Understanding the Input Layer

Data Preparation Code:

```
# Regression Example (Housing)
X_housing = housing.data  # 8 features
X_housing_scaled = StandardScaler().fit_transform(X_housing)

# Classification Example (Binary)
X_binary, _ = make_classification(n_features=2, n_redundant
    =0) #n_redundant: number of redundant features.
X_binary_scaled = StandardScaler().fit_transform(X_binary)

```

Weights and Biases

Understanding Weights and Biases

- **Weights (W)**
  - Connection strength between neurons
  - Learned during training
  - Matrix representation for each layer
- **Biases (b)**
  - Threshold for neuron activation
  - Adds flexibility to the model
  - Vector for each layer

Mathematical Representation

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g(z^{[l]})$$

where:

- $z^{[l]}$ is the weighted input to layer l
- $a^{[l]}$ is the activation of layer l
- $g()$ is the activation function

Weights and Biases Implementation

Code Implementation

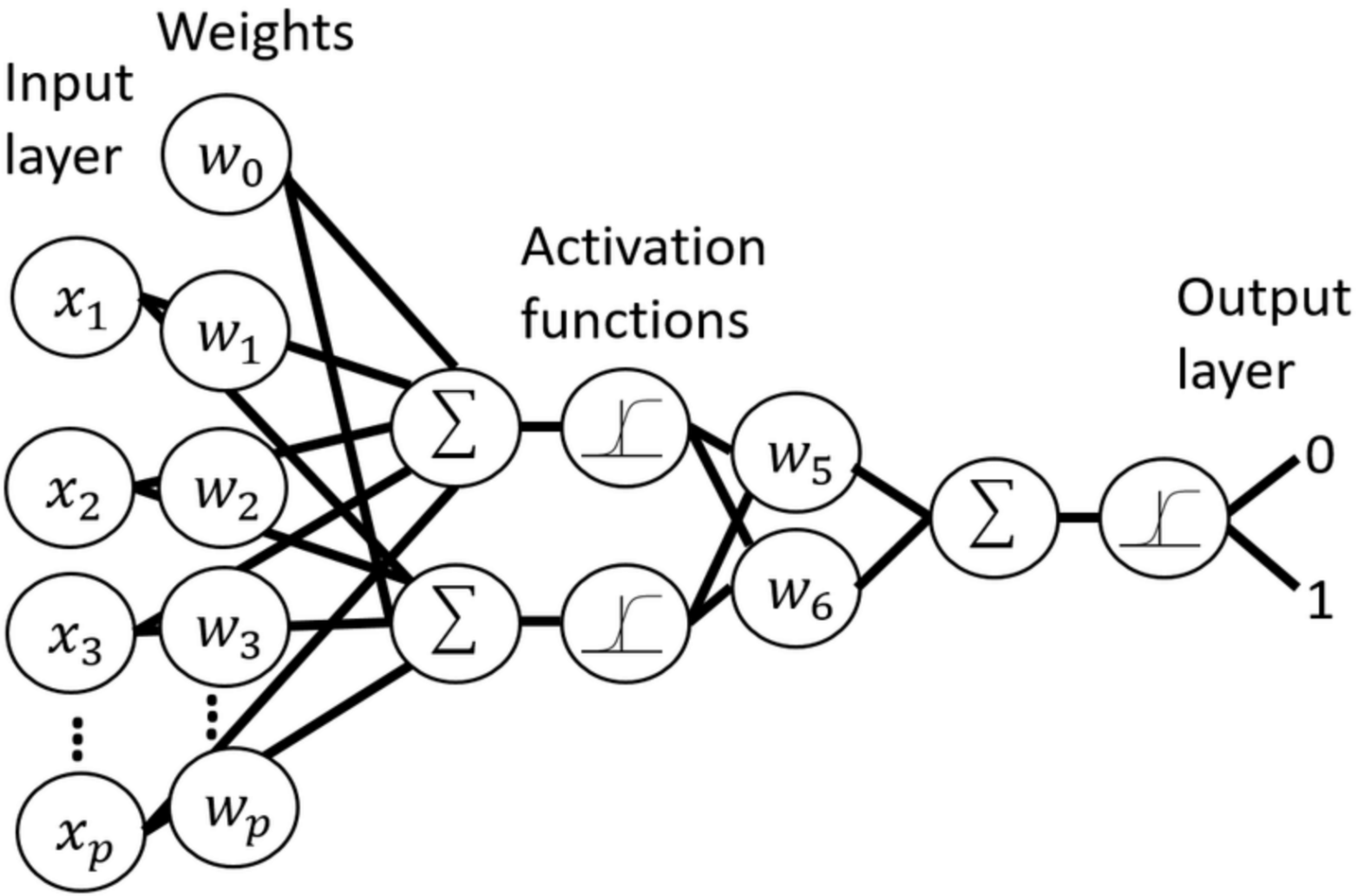
```
class NeuralNetworkLayer:
  def __init__(self, input_size, output_size):
  # Initialize weights using He initialization
    self.weights = np.random.randn(output_size, input_size)
    * np.sqrt(2/input_size)

    # Initialize biases to small values
      self.biases = np.zeros((output_size, 1))

  def forward(self, input_data):
  """Forward pass computation"""
    self.input = input_data
    # Compute weighted sum plus bias
    self.z = np.dot(self.weights, self.input) + self.biases
    # Apply activation function
    self.output = self.activation(self.z)
    return self.output

```

Neural Network Illustration



Common Activation Functions

- Sigmoid Function:

$$f(x) = \frac{1}{1 + e^{-x}}$$

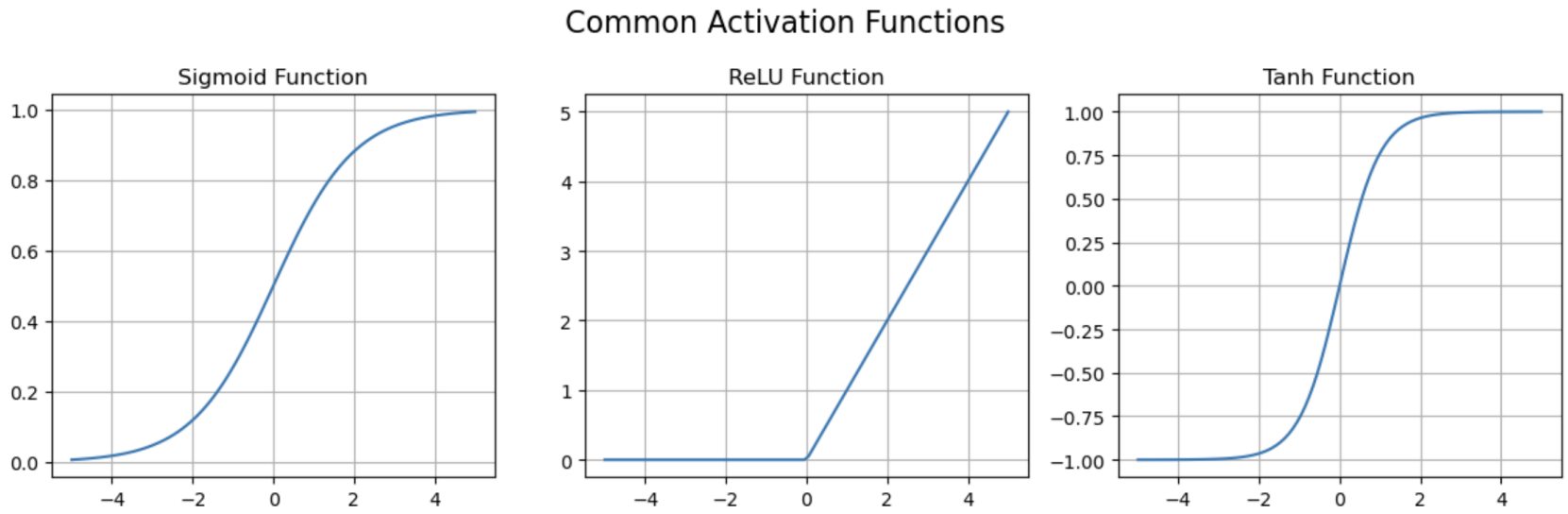
- ReLU Function:

$$f(x) = \max(0, x)$$

- Tanh Function:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Common Activation Functions



Activation Function Python Code

```python
# The @staticmethod decorator in Python is used to define a static method within a class. Static methods are functions that belong to the class but do not operate on a specific instance of the class or the class itself.

class ActivationFunctions:
  @staticmethod
  def sigmoid(x):
    return 1 / (1 + np.exp(-x))

  @staticmethod
  def relu(x):
    return np.maximum(0, x)

  @staticmethod
  def tanh(x):
    return np.tanh(x)


```

Hidden Layers - Part 1: Basic Concepts

Understanding Hidden Layers

- **Purpose**
  - Extract hierarchical features
  - Learn complex patterns
  - Transform data into useful representations
- **Components**
  - Input from previous layer
  - Weights and biases
  - Activation function
  - Output to next layer

Mathematical Flow For layer l:

$$\text{Input: } a^{[l-1]} \text{ (from previous layer)}$$
$$\text{Linear: } z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$
$$\text{Activation: } a^{[l]} = g(z^{[l]})$$

Hidden Layers - Part 2: Network Depth

Impact of Network Depth

- **Shallow Network (1-2 hidden layers)**

$$h_1 = g_1(W_1 x + b_1)$$
$$y = g_2(W_2 h_1 + b_2)$$

  - Simple feature extraction
  - Limited pattern recognition
  - Suitable for linear/simple problems

- **Deep Network (3+ hidden layers)**

$$h_1 = g_1(W_1 x + b_1)$$
$$h_2 = g_2(W_2 h_1 + b_2)$$
$$h_3 = g_3(W_3 h_2 + b_3)$$
$$y = g_4(W_4 h_3 + b_4)$$

  - Hierarchical feature learning
  - Complex pattern recognition
  - Better abstraction capability

Hidden Layers - Part 3: Feature Hierarchy

Layer-wise Feature Learning

- **First Hidden Layer**
  - Basic feature detection
  - Direct transformation of input
  - $h_1 = g(W_1 x + b_1)$
- **Middle Hidden Layers**
  - Combination of lower-level features
  - Abstract pattern formation
  - $h_l = g(W_l h_{l-1} + b_l)$
- **Deep Hidden Layers**
  - High-level feature composition
  - Complex pattern recognition
  - Increased abstraction

Considerations

- Vanishing/Exploding gradients in very deep networks
- Computational complexity increases with depth
- Need for skip connections in very deep networks

Hidden Layers - Usage Example

Training and Prediction Example

```
# Shallow network: Input -> Hidden -> Output (manual implementation)

def shallow_net_forward(x):
# x shape: (input_size, batch_size)
  z1 = np.dot(W1, x) + b1
  a1 = relu(z1)
  z2 = np.dot(W2, a1) + b2
  return z2  # Output (no activation for output layer here)
```

Hidden Layers - Usage Example

Training and Prediction Example

```
import torch.nn as nn

# Shallow network: Input -> Hidden -> Output
shallow_net = nn.Sequential(
nn.Linear(input_size, hidden_size),  # Layer 1
nn.ReLU(),                           # Activation
nn.Linear(hidden_size, output_size)  # Layer 2
  )
```

Hidden Layers - Usage Example

```
# Deep network: Input -> Hidden1 -> Hidden2 -> Hidden3 -> Output (manual implementation)


def deep_net_forward(x):
  # x shape: (input_size, batch_size)
  z1 = np.dot(W1, x) + b1
  a1 = relu(z1)
  z2 = np.dot(W2, a1) + b2
  a2 = relu(z2)
  z3 = np.dot(W3, a2) + b3
  a3 = relu(z3)
  z4 = np.dot(W4, a3) + b4
  return z4  # Output (no activation for output layer here)
```

Hidden Layers - Usage Example

Training and Prediction Example

```python
import torch.nn as nn

# Deep network: Input -> Hidden1 -> Hidden2 -> Hidden3 -> Output
deep_net = nn.Sequential(
            nn.Linear(input_size, hidden1_size),  # Layer 1
            nn.ReLU(),
            nn.Linear(hidden1_size, hidden2_size),# Layer 2
            nn.ReLU(),
            nn.Linear(hidden2_size, hidden3_size),# Layer 3
            nn.ReLU(),
            nn.Linear(hidden3_size, output_size)  # Output Layer
            )
```

Output Layer - Concepts

Output Layer Characteristics

- **Purpose**
  - Final transformation of features
  - Produces predictions in desired format
  - Maps to target variable space
- **Configuration by Task**
  - Regression: 1 neuron, linear activation
  - Binary Classification: 1 neuron, sigmoid activation
  - Multi-class: n neurons, softmax activation

Mathematical Formulation

$$\text{Regression: } y = W^{[L]}a^{[L-1]} + b^{[L]}$$

$$\text{Binary: } y = \sigma(W^{[L]}a^{[L-1]} + b^{[L]})$$

$$\text{Multi-class: } y = \text{softmax}(W^{[L]}a^{[L-1]} + b^{[L]})$$

Output Layer - Loss Functions

- **Regression**

$$\text{MSE: } J = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

$$\text{MAE: } J = \frac{1}{m}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

- **Classification**

$$\text{Binary Cross-Entropy: } J = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

$$\text{Categorical Cross-Entropy: } J = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{c} y_{ij}\log(\hat{y}_{ij})$$

Output Layer - Backpropagation

Gradient Flow

- **Chain Rule Application**

$$\frac{\partial J}{\partial W^{[L]}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{[L]}} \cdot \frac{\partial z^{[L]}}{\partial W^{[L]}}$$

$$\frac{\partial J}{\partial b^{[L]}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{[L]}}$$

- **Error Propagation**
  - Output error: $\delta^{[L]} = \frac{\partial J}{\partial \hat{y}} \cdot g'(z^{[L]})$
  - Hidden layer error: $\delta^{[l]} = (W^{[l+1]T}\delta^{[l+1]}) \cdot g'(z^{[l]})$
- **Parameter Updates**

$$W^{[l]} = W^{[l]} - \alpha \cdot \delta^{[l]} a^{[l-1]T}$$

$$b^{[l]} = b^{[l]} - \alpha \cdot \delta^{[l]}$$

Output Layer - Implementation

```
def forward(self, X):
  self.input = X
  self.z = np.dot(self.weights, X) + self.biases

  if self.task == 'regression':
    self.output = self.z  # Linear activation
  elif self.task == 'binary':
    self.output = 1 / (1 + np.exp(-self.z))  # Sigmoid
  else:  # multiclass
    exp_z = np.exp(self.z - np.max(self.z))
    self.output = exp_z / np.sum(exp_z, axis=0)
  return self.output

```

Backpropagation - Implementation

```python
def backward(self, y_true, learning_rate=0.01):
  m = y_true.shape[1]

  if self.task == 'regression':
    self.dz = (self.output - y_true) / m
  elif self.task == 'binary':
    self.dz = (self.output - y_true) / (m * self.output * (1 - self.output))
  else:  # multiclass
    self.dz = (self.output - y_true) / m

  # Compute gradients
  self.dW = np.dot(self.dz, self.input.T)
  self.db = np.sum(self.dz, axis=1, keepdims=True)

  # Update parameters
  self.weights -= learning_rate * self.dW
  self.biases -= learning_rate * self.db

  # Return gradient for previous layer
  return np.dot(self.weights.T, self.dz)

```

California Housing Dataset Overview

Dataset Description

- **Source**
  - Based on 1990 California census data
  - Derived from U.S. Census Bureau data
  - Geographic data across California districts
- **Dataset Characteristics**
  - Instances: 20,640 districts
  - Features: 8 numeric predictive attributes
  - Target Variable: Median house value
  - No missing values

Dataset Features and Goal

Feature Description

- **Geographical Features**
  - Longitude: District's longitude position
  - Latitude: District's latitude position
- **Housing Features**
  - MedInc: Median income in block (in tens of thousands)
  - HouseAge: Median house age in block
  - AveRooms: Average number of rooms
  - AveBedrms: Average number of bedrooms
  - Population: Block population
  - AveOccup: Average house occupancy
- **Target Variable**
  - Median house value (in hundreds of thousands)
  - Range: $14,999 to $500,001

Regression Goal

- Predict median house values based on district features
- Learn relationships between location, demographics, and price
- Understand factors influencing California housing market

Dataset Loading and Exploration

```python
from sklearn.datasets import fetch_california_housing
import pandas as pd
import numpy as np

# Load the dataset
housing = fetch_california_housing()
X = housing.data
y = housing.target

# Create a pandas DataFrame
df = pd.DataFrame(X, columns=housing.feature_names)
df['Price'] = y

# Basic statistics
print("Dataset Shape:", X.shape)
print("\nFeature Statistics:")
print(df.describe())

```

Data Preparation

```
# Load and split data
housing = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

Model Configuration

```
model = MLPRegressor(
    hidden_layer_sizes=(100, 50),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42,
    learning_rate_init=0.001
    )

```

- Two hidden layers (100, 50 neurons)
- ReLU activation
- Adam optimizer
- 1000 maximum iterations

Model Evaluation

Performance Metrics

- Mean Squared Error (MSE)

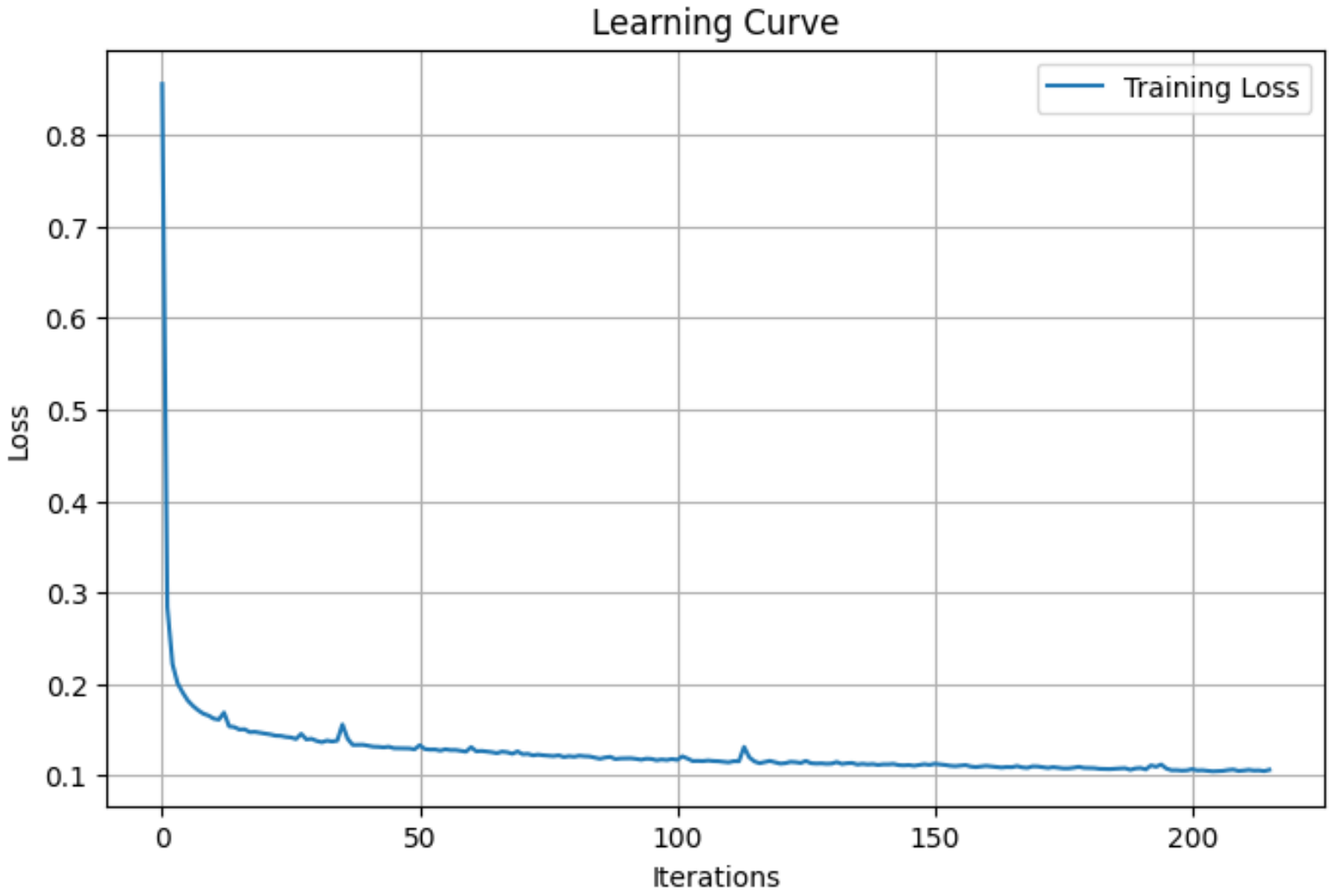
$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- R^2 Score

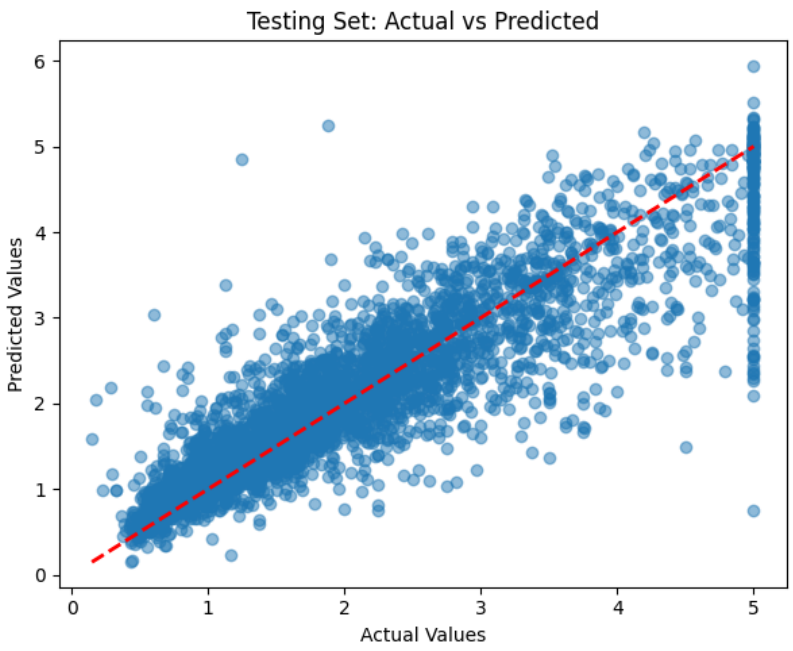
$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- Visualization
  - Actual vs Predicted plots
  - Learning curves

Loss Function



Regression Result



Classification Overview

Types of Classification

- **Binary Classification**
  - Two possible classes (0 or 1)
  - Examples: Spam detection, Disease diagnosis
  - Output layer: Single neuron with sigmoid
- **Multi-class Classification**
  - Multiple possible classes
  - Examples: Digit recognition, Image classification
  - Output layer: n neurons with softmax

Classification Implementation

```
# Generate classification dataset
X, y = make_classification(
    n_samples=1000,
    n_features=2,
    n_redundant=0,
    n_informative=2,
    random_state=42,
    n_clusters_per_class=1
    )
y = y.reshape(-1, 1)

# Split and scale data
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)

```

Neural Network Configuration

```
# Define and train a simple ANN classifier
clf = MLPClassifier(
        hidden_layer_sizes=(100, 50),
        activation='relu',
        solver='adam',
        max_iter=1000,
        random_state=42
        )
clf.fit(X_train, y_train)

```

Correct Predictions: TP and TN

- **TP (True Positive)**
  - **Explanation:** The model correctly predicts the positive class. In other words, the test correctly identifies the presence of a specific condition when it is actually present.
  - **Example:** A spam filter correctly identifies a spam email and sends it to the spam folder.

- **TN (True Negative)**
  - **Explanation:** The model correctly predicts the negative class. This means the test accurately identifies the absence of a condition.
  - **Example:** A legitimate email is correctly identified as not spam and is sent to the inbox.

Incorrect Predictions: FP and FN

- **FP (False Positive)**
  - **Explanation:** The model incorrectly predicts the positive class when it is actually negative. This is also known as a "Type I error".
  - **Example:** A legitimate email is incorrectly classified as spam. This is often called a "false alarm".

- **FN (False Negative)**
  - **Explanation:** The model incorrectly predicts the negative class when it is actually positive. This is also known as a "Type II error".
  - **Example:** A spam email is missed by the filter and goes to the main inbox.

Classification Performance Metrics

Evaluation Metrics

- Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision

$$Precision = \frac{TP}{TP + FP}$$

- Recall

$$Recall = \frac{TP}{TP + FN}$$

- F1 Score

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Visualization Code

```
# Predict and evaluate
y_pred_proba = clf.predict_proba(X_train)[:, 1]
train_loss = log_loss(y_train, y_pred_proba)
y_pred = clf.predict(X_train)
train_acc = accuracy_score(y_train, y_pred)
print(f"Train Loss: {train_loss:.4f}, Train Accuracy: {train_acc:.4f}")

y_test_pred_proba = clf.predict_proba(X_test)[:, 1]
test_loss = log_loss(y_test, y_test_pred_proba)
y_test_pred = clf.predict(X_test)
test_acc = accuracy_score(y_test, y_test_pred)
print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_acc:.4f}")

#plot the loss curve
plt.plot(clf.loss_curve_)
plt.show()

```

Loss Function

References

- Neural Networks and Deep Learning by Michael Nielsen
- Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville
- Scikit-learn Documentation
- GeeksforGeeks ANN Tutorial

Thank You !